Supplementary Document for the Polycenter Algorithm

Yake, Zhai

Henan Jiahe Technology Co., Ltd, Fengtai community, Shenbo Road, Nanyang, China

ARTICLE HISTORY
Compiled May 26, 2025

ABSTRACT

This supplementary document provides detailed information regarding the implementation of the Polycenter algorithm in C++. The algorithm is designed to compute the visual center of any polygon, which corresponds to the center of the largest inscribed circle, and is applicable in various domains, including robotics, computer vision, computational biology, and so on.

KEYWORDS
Visual center, Maximum inner circle, Polygons with holes

1. Overview

The Polycenter algorithm is capable of handling simple polygons, including convex, concave, and polygons with holes. It does not support self-intersecting polygons. (see in Fig.1)

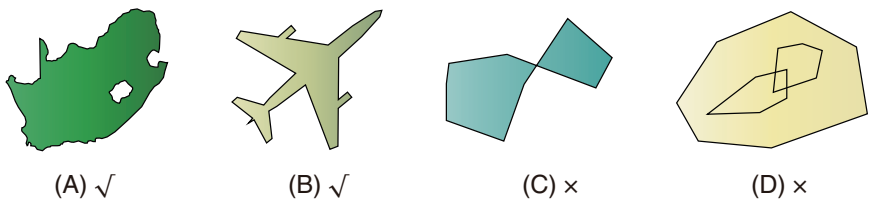

Figure 1. The scope of Polycenter

The core functionality is encapsulated within the *Polycenter.hpp* file, which can be utilized in a C++ 14 compliant environment with the following syntax:

$$auto\ result = Polygon(pts).maximumCircle();$$

where *pts* represents the list of vertices for each polygon loop, with the outer boundary in counterclockwise order and holes in clockwise order. The output includes the coordinates of the visual center and the radius of the maximum circle.

CONTACT Yake, Zhai. Email: yakezhai@gmail.com

2. Test the program

In order to facilitate the reproduction of our experimental results by the research community, we have made available two testing methodologies. The first is a ready-to-run executable (exe) file, which can be directly executed without additional setup. The second is a plug-in designed to be integrated into AutoCAD 2024, providing a platform-specific means of conducting the tests as described in our study.

2.1. *Directly executed*

As illustrated in Fig.2, locate *Sample.exe* within the *Code* folder of the supplementary materials and copy it to the folder containing the test data. Double-click on the executable file, and upon the command prompt window appearing, enter the number of repetitions for the computation (it is recommended to input 1 during the initial test). The program will display the computational results. It should be noted that when testing within the shapes folder, due to the limitations of Polylabel, deadlocks may occur during the processing of *Cutting board* and *Rectangle* shapes due to parallel edges of polygons, resulting in severe computation timeouts. To avoid interference from these two shapes, please delete them from the folder.

Furthermore, *Sample.cpp* provides the source code for Sample.exe, where both Polylabel and Polycenter employ the same mechanism within the code to ensure fairness in testing.

2.2. *Test in AutoCAD2024*

In contrast to the first approach, the testing procedure within AutoCAD 2024 is more intuitive. Follow these steps to initiate the test:

(1) Open AutoCAD 2024: Ensure you are using the specific version number (AutoCAD 2024) to avoid compatibility issues.
(2) Load the Plugin: Use the *appload* command to load the *Polycenter.arx* plugin. Upon successful loading, two new commands, *Polycenter* and *Polylabel*, will be integrated into AutoCAD.
(3) Execute the *Polycenter* Command: Run the Polycenter command, input the desired number of iterations for recalculation, select a closed polygon, and press Enter to observe the operational outcome.

Note: To evaluate polygons with holes, you must simultaneously select both the outer boundary and all enclosed holes. The software will automatically identify the outermost wireframe as the polygon's boundary and the internal wireframes as the holes.

2.3. *Test in OpenJUMP*

To facilitate comparison with MICGIS, we also provide a version compatible with OpenJUMP. Before testing, copy the *Polycenter.dll* file to OpenJUMP's *\lib\native* folder and the *Polycenter.jar* file to the *\lib\ext* folder. After launching OpenJUMP, two new submenus - *Polycenter* and *Polylabel* - will be available under the Plugins menu.

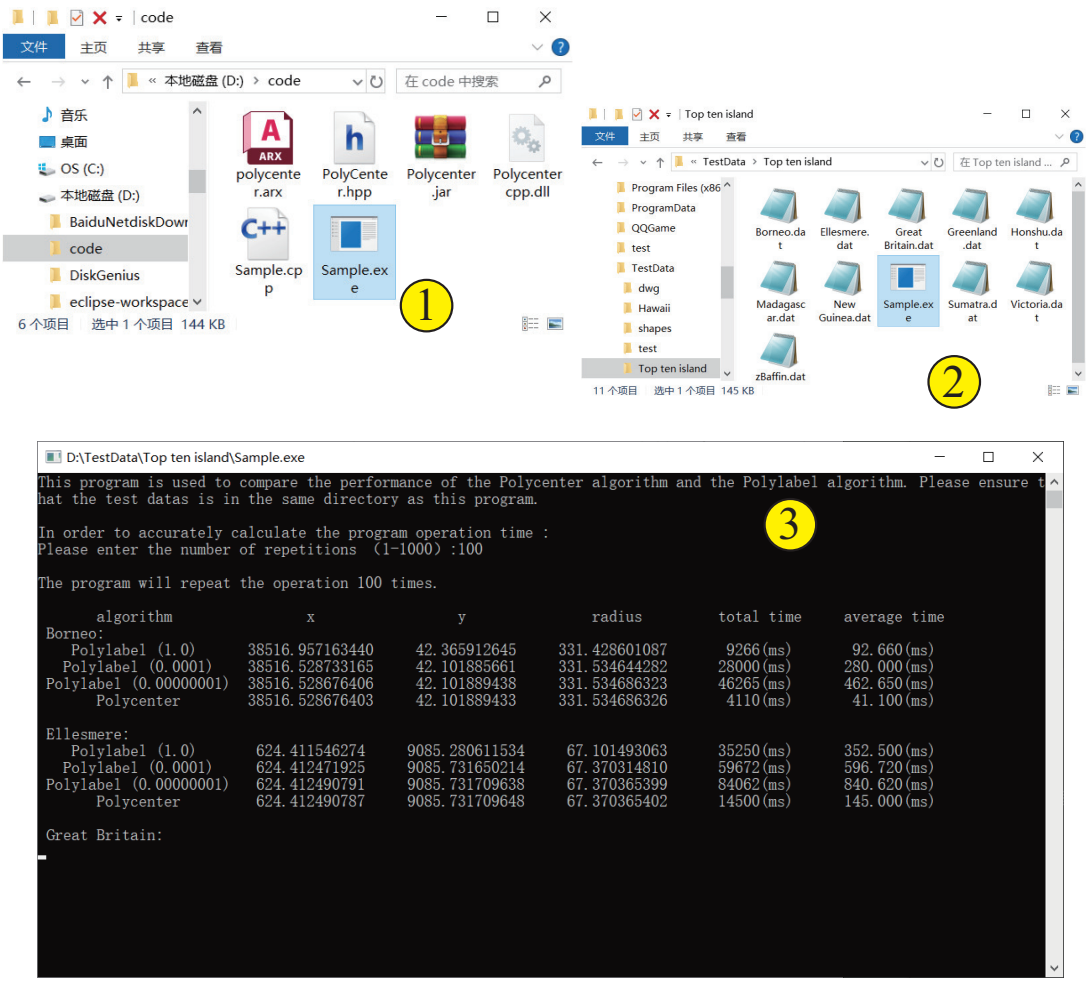

Figure 2. directly executed

Click the menu and input the required parameters to proceed. Note: As demonstrated in Fig.4, OpenJUMP may require extended processing time to render circular features when handling large datasets. Allow sufficient time for the operation to complete.

3. Example Data

This package provides example datasets, which contain the outlines of the ten largest islands, formatted in a straightforward binary structure.

Figure 5 illustrates the file's architecture: it commences with an integer indicating the number of loops, followed by the number of vertices in each loop, and then the sequential coordinates of these vertices. The external boundary is recorded in a counterclockwise direction, while internal holes are traced clockwise. This method facilitates a structured and computationally efficient encoding of the geometric data. For ease of visual verification, the dataset is also supplied with files in the DWG format.

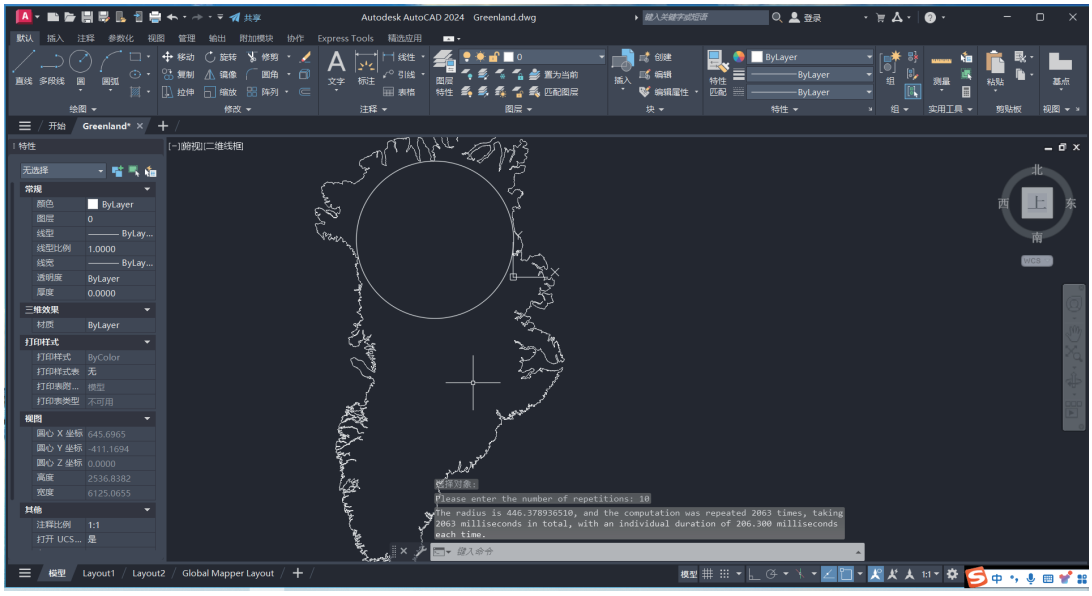

Figure 3. Test Polycenter in AutoCAD 2024

4. Test Results

4.1. *Top 10 Largest Islands*

As presented in the paper, direct comparisons between the Polycenter and Polylabel algorithms were conducted on the top 10 largest islands. To enable MICGIS participation in the comparison, both algorithms were implemented within MICGIS-supported platforms (OpenJUMP). The comparative results are shown below:

4.2. *Hawaiian Archipelago*

Similarly, comparative results for the Hawaiian Islands are as follows:

4.3. *Continents*

To stress-test the algorithms, we utilized the outer contours of seven continents. Due to MICGIS's prohibitively long processing times (e.g., 15 days for Greenland's computation), it was excluded from this comparison.

4.4. *Synthetic Polygons*

For large-scale evaluation, we created complex polygons with holes by combining the seven continents and eight Hawaiian Islands. The comparative results are shown below:

Table 1. Top ten Largest Islands Outer Contour Maximum Circle Algorithm Area

Name	Sides	Time-consuming (ms)					Maximum radius	
		Polylabel			MICGIS	Poly center	MICGIS	Polycenter
		1.0	10^{-4}	10^{-8}				
Greenland	650162	960	1616	2362	> 15 days	298	-	446.378936510
New Guinea	135248	94	430	765	$5.88 \times 10^7 (16.3h)$	36	305.616859296	305.616859296
Borneo	145975	108	308	616	$5.88 \times 10^7 (16.3h)$	70	331.534686326	331.534686326
Madagascar	65086	32	91	150	$7.42 \times 10^7 (20.6h)$	22	106.991629368	106.991629368
Baffin	312770	340	673	1009	$2.56 \times 10^8 (71.0h)$	175	127.104876761	127.104876761
Sumatra	38092	60	202	315	$2.98 \times 10^6 (0.8h)$	22	167.752197858	167.752197858
Honshu	183733	209	678	1187	$7.56 \times 10^6 (2.1h)$	122	114.946024290	114.946024290
Victoria	91013	64	145	227	$1.79 \times 10^7 (5.0h)$	33	124.918610305	124.918610305
Great Britain	223395	220	473	675	$4.37 \times 10^8 (121.3h)$	119	120.072346685	120.072346685
Ellesmere	266744	415	630	850	$1.93 \times 10^8 (53.6h)$	254	67.370365402	67.370365402

Table 2. Hawaii Main Islands Outer Contour Maximum Circle Algorithm Area

Name	Sides	Time-consuming (ms)					Maximum radius	
		Polylabel			MICGIS	Polycenter	MICGIS	Polycenter
		1.0	10^{-4}	10^{-8}				
Hawaii	7664	3.0	12.6	24.7	184011	2.1	46.259058907	46.259058907
Maui	4577	1.4	5.4	8.9	49794	1.1	17.074115144	17.074115144
Oahu	4642	1.5	4.6	7.1	55709	1.5	13.962469612	13.962469612
Kauai	3538	0.8	8.5	16.4	24048	1.0	17.414611661	17.414611661
Molokai	3369	1.9	7.0	10.8	24123	1.7	6.275565036	6.275565036
Lanai	1523	0.4	1.2	2.2	8372	0.3	8.440127842	8.440127842
Nihau	1363	0.5	7.1	13.7	9193	0.9	3.981586152	3.981586152
kahoolawe	1102	0.3	1.0	1.9	5619	0.4	3.902256007	3.902256007

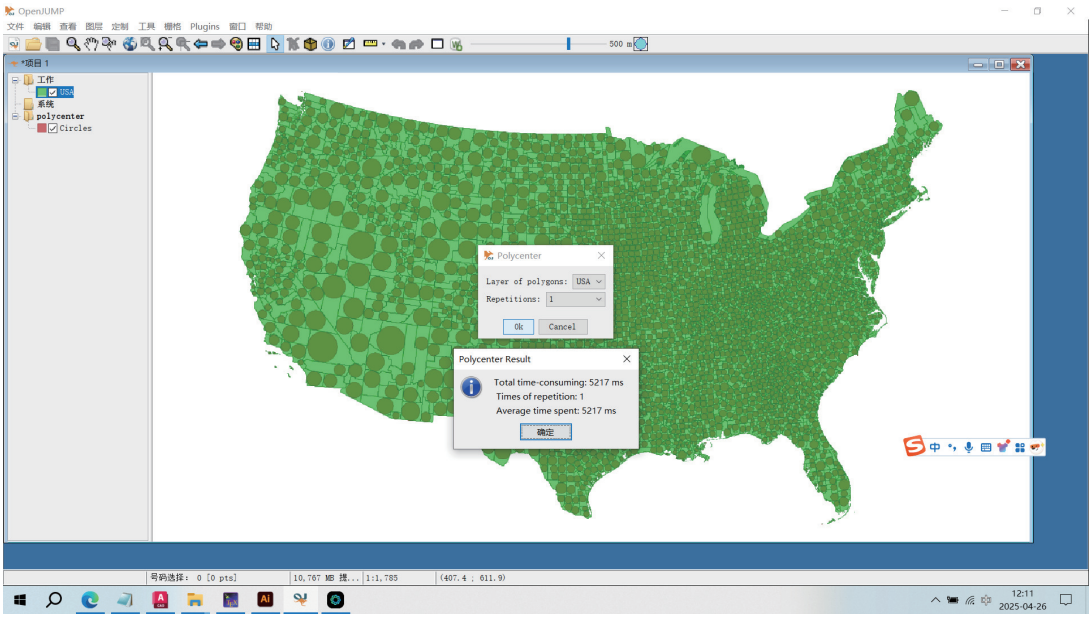

Figure 4. Test Polycenter in OpenJUMP

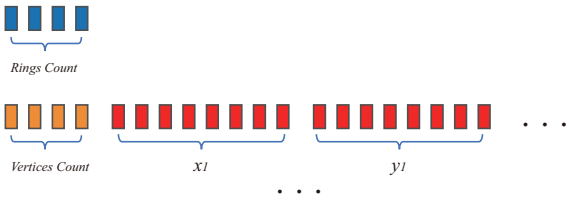

Figure 5. File structure: Sequentially store each ring.

Table 4. Synthetic Polygons Maximum Circle Algorithm Area

Name	Sides	Maximum-radius\Time-consuming (ms)			
		Polylabel			Polycenter
		1.0	0.0001	0.00000001	
Virtual Hawaii	27778	13.132626811\47	13.480407910\94	13.480428540\156	13.480428542\28
Virtual Islands	2112248	185.208025029\4968	185.529023713\9234	185.529055539\12484	185.529055542\6594
Virtual World	10837708	20.316076576\14453	20.478442777\29578	20.478466296\44563	20.478466298\23422

4.5. *Case Study: U.S. Administrative Regions*

As a practical case, we tested over 8,000 U.S. administrative regions and islands. The computation time comparison is illustrated below:

5. Class Definitions

The program utilizes several classes to represent polygon elements:

(1) **Vertex:** Represents a vertex and its connection to adjacent vertices. In addition, it also records the side formed by it and its next.

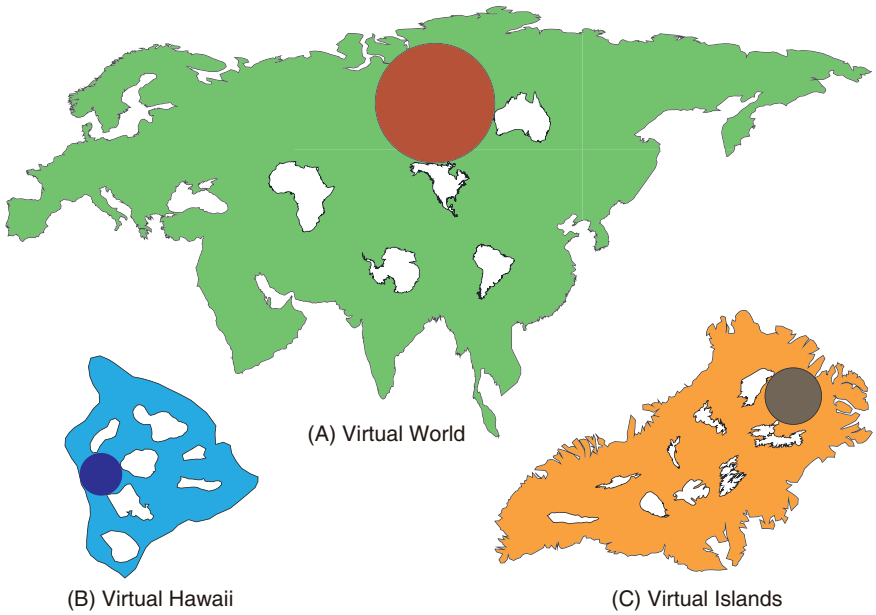

Figure 6. Synthetic Polygons

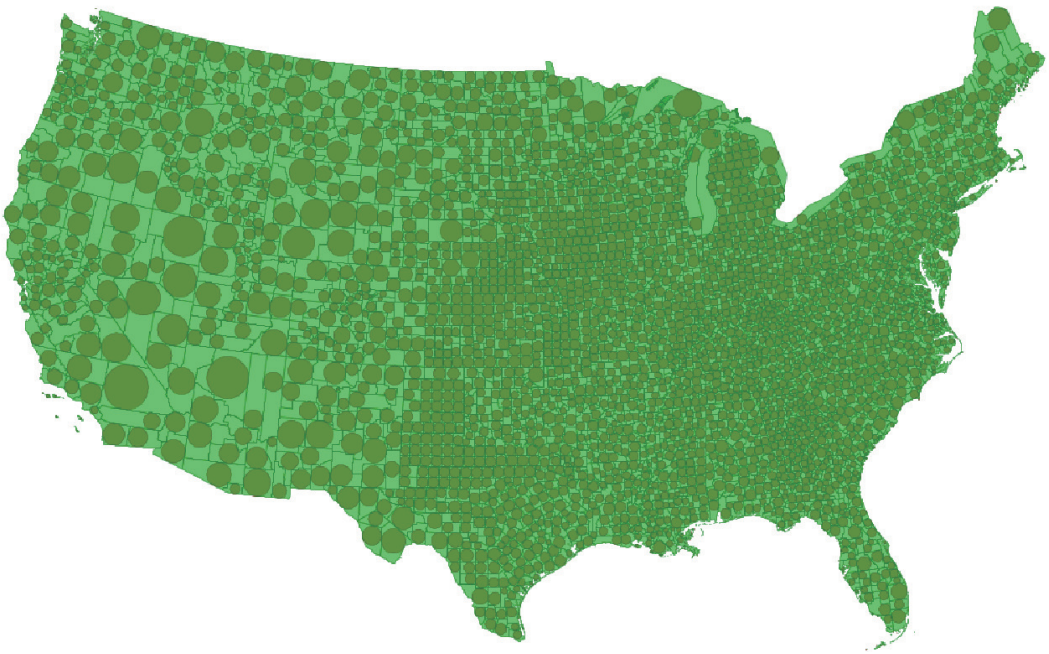

Figure 7. Case Study

Table 3. Continents Outer Contour Maximum Circle Algorithm Area

Name	Sides	Maximum-radius\Time-consuming (ms)			
		Polylabel			Polycenter
		1.0	0.0001	0.00000001	
Africa	1264416	16.313691342\594	16.418566131\2218	16.418572567\3734	16.418572570\516
Antarctica	703124	1177.773954674\610	1177.864627959\1344	1177.864651026\2031	1177.864651027\156
Eurasia	4239319	24.870194365\2187	24.902670822\6610	24.902713417\10750	24.902713418\1188
North America	2615934	16.419842811\1125	16.658720626\4938	16.658756397\8734	16.658756398\891
Oceania	570488	8.234755632\188	8.418049669\1328	8.418068489\2188	8.418068492\234
South America	1444427	14.474032797\563	14.726534382\1875	14.726568983\3031	14.726568985\437

(2) **Ring:** Represents a loop forming the polygon's boundary or holes. Since vertex can be connected to each other, the Ring only needs to record any vertex. In the program, if the Ring is divided from the higher-level polygon, we record the starting vertex of the stitching side.
(3) **Position** Represents any point on the polygon.

Key classes for calculation include:

(1) **IntervalsforL:** Records the state of interval L and is used to add new constraints when they are generated by the sides or vertices.
(2) **Datumline :** Used to search for the extreme circle on a line and return the Chains divided by the circle

Additionally, some global variables are used to record the largest circle that has been discovered so far.

6. Filter

In the associated paper, we utilized the methodology illustrated in Fig.8 to effectively filter out invalid edges and vertices. The processes denoted as A and C are relatively straightforward, whereas B and D represent more intricate procedures. Owing to space constraints in the main text, a comprehensive elucidation of these methodologies was not provided. In this supplementary document, we aim to furnish a detailed supplementary explanation focusing on points B and D to enhance the reader's understanding.

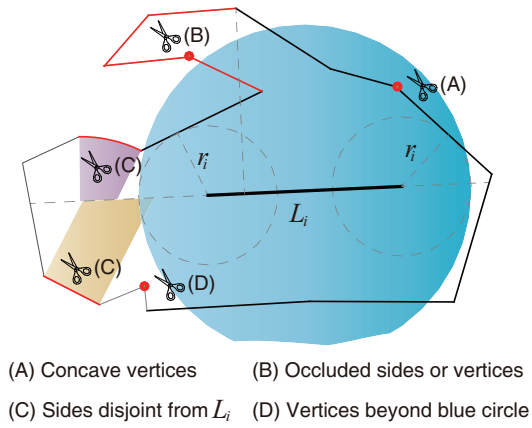

Figure 8. Filter: With circle growth, elements lacking new constraints are progressively filtered out.

6.1. *Occlusion Filtering*

This section provides a detailed explanation of the algorithm used to filter out occluded elements during the calculation process, including the logic behind the *searchtoNext* function of the Line class.

The ring frequently intersects the line at its starting side, which is typically the sewing side. Upon this side, the labeling process commences. This process moving downward and assigning labels to each side according to its bend, as depicted in Fig.9. We keep a record of sides moving in the designated forward direction (code 1) and modify this record by adding sides moving in the reverse direction (code 2) as we proceed. After traversing the entire structure, we acquire an optimized list of sides.

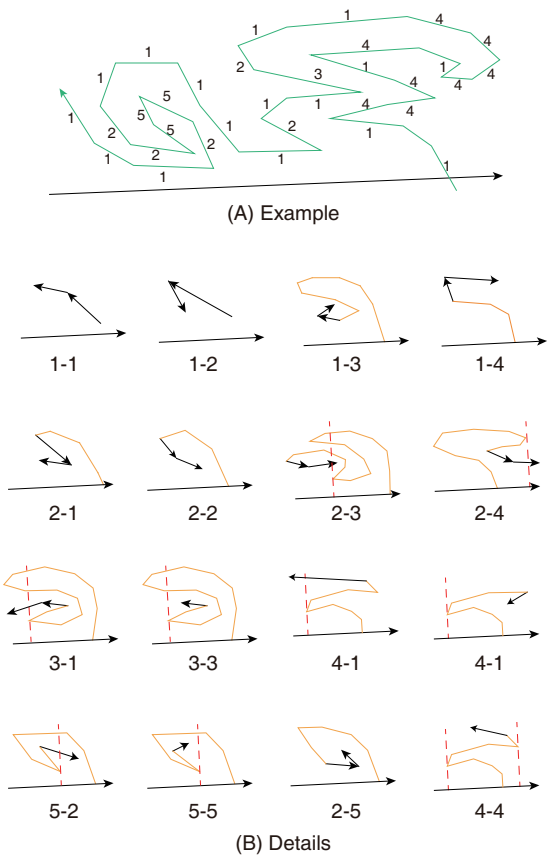

Figure 9. Occlusion Filtering

For other rings, due to the increased system consumption from searching for the first side that intersects with the line and the possibility that the entire ring does not intersect with the line, requiring additional processing, after careful consideration, we opt not to use the aforementioned method. Instead, we directly add the sides moving forward to the list of valid sides.

Subsequently, we traverse the side list, evaluate the endpoints of each, filter out the concave vertices, and compile a valid vertex list.

6.2. *Vertices Filtering*

During the L-narrowing procedure, we utilized blue circles to eliminate invalid vertices. The justification for this approach is based on the observation that all inner circles, constrained by L, are entirely enclosed within the blue circle Consequently, vertices outside the blue circle cannot intersect with these inner circles and are therefore considered invalid. To validate this claim, we proceed with a proof by contradiction:

It is established that in the i-th constraint,L defines a radius (r_i). As a result, a circle with radius (r_i), centered at the endpoint of L, must necessarily contact the polygon at a point. If this were not the case, L could be extended, which would contradict its defining criteria. Any circle constrained by L and extending beyond the boundary of the blue circle would have to fully enclose either the left or right inner circle. In doing so, it would also include the aforementioned contact point, which lies on the polygon. This scenario contradicts the definition of an inner circle, thereby confirming the correctness of our inference.

7. Split and close

The polygon's segmentation and closure are implemented in the class Clipper within the code. It may appear complex because it has to handle some intricate situations, as shown in Fig.10.

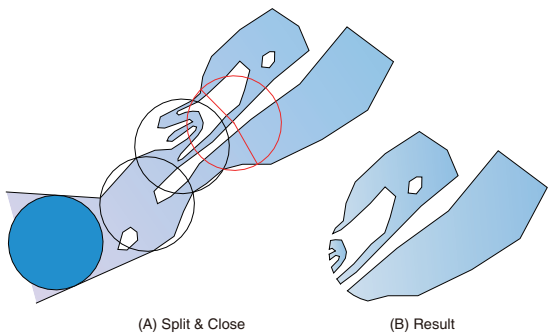

Figure 10. Sub-polygon

During the closure process, for a single loop, we directly use the *break* of the corresponding *position* to form a new *vertex*, and then connect the vertices using the *above* and *below* properties of the Vertex, thus completing the segmentation. For situations with multiple loops, we need to handle them separately and sort the cutting points. Then, we close each pair of cutting points to form multiple new loops. Finally, we must determine the ownership of the loops to ascertain which hole belongs to which sub-polygon.

8. Floating-Point Handling Strategy

Given that the algorithm involves extensive floating-point computations, a tolerance threshold $\zeta = 10^{-10}$ is implemented to mitigate numerical uncertainties. Two regular floating-point numbers are considered identical when their difference falls below $< \zeta$. For near-parallel condition handling, $\xi = 10^{-8}$ serves as the reference criterion for radius invariance.

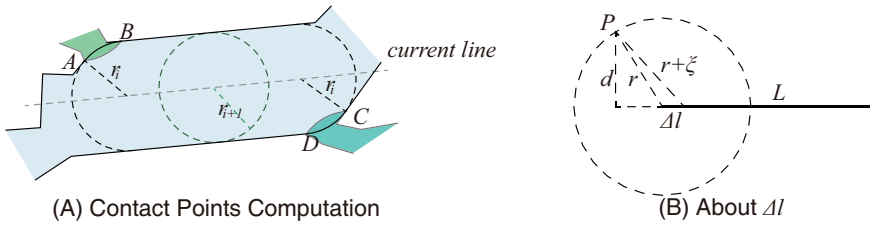

Figure 11. Floating-Point Handling Strategy

In boundary determination of set L derived from constrained elements, when evaluating circle-polygon contacts using the dual semicircle contact method proposed in the paper, we introduce the following quantitative criterion (as illustrated in Fig.11): The variation Δl in L is calculated as $\Delta l = \sqrt{(r+\xi)^2 - d^2} - \sqrt{r^2 - d^2}$.

As d approaches $r(d \to r)$, $\Delta l \approx \sqrt{2r\xi}$, while in other $\Delta l \approx r\xi/\sqrt{r^2 - d^2}$.

A comprehensive criterion $\Delta l < \sqrt{2r\xi}$ is adopted to determine L's invariance, ensuring robust numerical stability across geometric configurations.

9. Comparison algorithms

The comparison algorithms mentioned in the paper are all accessible on GitHub. Should you require them, you can find them at the following URLs:

To utilize the Polylabel algorithm, please visit https://github.com/mapbox/polylabel. Note that when implementing it, you will also need to include its dependency, mapbox.

For Beyhan's algorithm, please visit https://github.com/burakbeyhan/maximum-inscribed-circle. Moreover, you may need to download OpenJUMP to execute this algorithm.

For your convenience, we have pre-downloaded the latest versions of these algorithms and included them in the file package accompanying this document.